Minimizing the average squared reconstruction error is equivalent to minimizing the projection of the data covariance matrix onto the orthogonal complement of the principal subspace.

Minimizing the average squared reconstruction error is equivalent to maximizing the variance of the projected data.

projected onto the orthogonal complement of the principal subspace. Minimizing the average squared reconstruction error is therefore equivalent to minimizing the variance of the data when projected onto the subspace we ignore, i.e., the orthogonal complement of the principal subspace. Equivalently, we maximize the variance of the projection that we retain in the principal subspace, which links the projection loss immediately to the maximum-variance formulation of PCA discussed in Section 10.2. But this then also means that we will obtain the same solution that we obtained for the maximum-variance perspective. Therefore, we omit a derivation that is identical to the one presented in Section 10.2 and summarize the results from earlier in the light of the projection perspective.

The average squared reconstruction error, when projecting onto the M-dimensional principal subspace, is

$$J_M = \sum_{j=M+1}^{D} \lambda_j \,, \tag{10.44}$$

where λ_j are the eigenvalues of the data covariance matrix. Therefore, to minimize (10.44) we need to select the smallest $D - M$ eigenvalues, which then implies that their corresponding eigenvectors are the basis of the orthogonal complement of the principal subspace. Consequently, this means that the basis of the principal subspace comprises the eigenvectors $\boldsymbol{b}_1, \ldots, \boldsymbol{b}_M$ that are associated with the largest M eigenvalues of the data covariance matrix.

Example 10.3 (MNIST Digits Embedding)

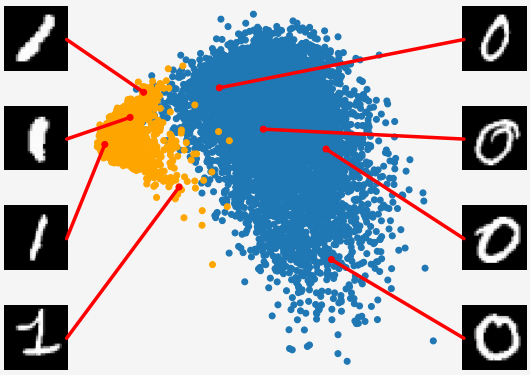


Figure 10.10 Embedding of MNIST digits 0 (blue) and 1 (orange) in a two-dimensional principal subspace using PCA. Four embeddings of the digits "0" and "1" in the principal subspace are highlighted in red with their corresponding original digit.

Figure 10.10 visualizes the training data of the MMIST digits "0" and "1" embedded in the vector subspace spanned by the first two principal components. We observe a relatively clear separation between "0"s (blue dots) and "1"s (orange dots), and we see the variation within each individual

cluster. Four embeddings of the digits "0" and "1" in the principal subspace are highlighted in red with their corresponding original digit. The figure reveals that the variation within the set of "0" is significantly greater than the variation within the set of "1".

10.4 Eigenvector Computation and Low-Rank Approximations

In the previous sections, we obtained the basis of the principal subspace as the eigenvectors that are associated with the largest eigenvalues of the data covariance matrix

$$\boldsymbol{S} = \frac{1}{N}\sum_{n=1}^{N} \boldsymbol{x}_n \boldsymbol{x}_n^\top = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top, \tag{10.45}$$

$$\boldsymbol{X} = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_N] \in \mathbb{R}^{D\times N}. \tag{10.46}$$

Note that $\boldsymbol{X}$ is a $D \times N$ matrix, i.e., it is the transpose of the "typical" data matrix (Bishop, 2006; Murphy, 2012). To get the eigenvalues (and the corresponding eigenvectors) of $\boldsymbol{S}$, we can follow two approaches:

Use eigendecomposition or SVD to compute eigenvectors.

- We perform an eigendecomposition (see Section 4.2) and compute the eigenvalues and eigenvectors of $\boldsymbol{S}$ directly.
- We use a singular value decomposition (see Section 4.5). Since $\boldsymbol{S}$ is symmetric and factorizes into $\boldsymbol{X}\boldsymbol{X}^\top$ (ignoring the factor $\frac{1}{N}$), the eigenvalues of $\boldsymbol{S}$ are the squared singular values of $\boldsymbol{X}$.

  More specifically, the SVD of $\boldsymbol{X}$ is given by

$$\underbrace{\boldsymbol{X}}_{D\times N} = \underbrace{\boldsymbol{U}}_{D\times D}\underbrace{\boldsymbol{\Sigma}}_{D\times N}\underbrace{\boldsymbol{V}^\top}_{N\times N}, \tag{10.47}$$

where $\boldsymbol{U} \in \mathbb{R}^{D\times D}$ and $\boldsymbol{V}^\top \in \mathbb{R}^{N\times N}$ are orthogonal matrices and $\boldsymbol{\Sigma} \in \mathbb{R}^{D\times N}$ is a matrix whose only nonzero entries are the singular values $\sigma_{ii} \geqslant 0$. It then follows that

$$\boldsymbol{S} = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top = \frac{1}{N}\boldsymbol{U}\boldsymbol{\Sigma}\underbrace{\boldsymbol{V}^\top\boldsymbol{V}}_{=\boldsymbol{I}_N}\boldsymbol{\Sigma}^\top\boldsymbol{U}^\top = \frac{1}{N}\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^\top\boldsymbol{U}^\top. \tag{10.48}$$

With the results from Section 4.5, we get that the columns of $\boldsymbol{U}$ are the eigenvectors of $\boldsymbol{X}\boldsymbol{X}^\top$ (and therefore $\boldsymbol{S}$). Furthermore, the eigenvalues λ_d of $\boldsymbol{S}$ are related to the singular values of $\boldsymbol{X}$ via

The columns of $\boldsymbol{U}$ are the eigenvectors of $\boldsymbol{S}$.

$$\lambda_d = \frac{\sigma_d^2}{N}. \tag{10.49}$$

This relationship between the eigenvalues of $\boldsymbol{S}$ and the singular values of $\boldsymbol{X}$ provides the connection between the maximum variance view (Section 10.2) and the singular value decomposition.

©2024 M. P. Deisenroth, A. A. Faisal, C. S. Ong. Published by Cambridge University Press (2020).

10.4.1 PCA Using Low-Rank Matrix Approximations

To maximize the variance of the projected data (or minimize the average squared reconstruction error), PCA chooses the columns of $\boldsymbol{U}$ in (10.48) to be the eigenvectors that are associated with the M largest eigenvalues of the data covariance matrix $\boldsymbol{S}$ so that we identify $\boldsymbol{U}$ as the projection matrix $\boldsymbol{B}$ in (10.3), which projects the original data onto a lower-dimensional subspace of dimension M. The *Eckart-Young theorem* (Theorem 4.25 in Section 4.6) offers a direct way to estimate the low-dimensional representation. Consider the best rank-M approximation

Eckart-Young theorem

$$\tilde{\boldsymbol{X}}_M := \operatorname{argmin}_{\operatorname{rk}(\boldsymbol{A}) \leqslant M} \|\boldsymbol{X} - \boldsymbol{A}\|_2 \in \mathbb{R}^{D \times N} \tag{10.50}$$

of $\boldsymbol{X}$, where $\|\cdot\|_2$ is the spectral norm defined in (4.93). The Eckart-Young theorem states that $\tilde{\boldsymbol{X}}_M$ is given by truncating the SVD at the top-M singular value. In other words, we obtain

$$\tilde{\boldsymbol{X}}_M = \underbrace{\boldsymbol{U}_M}_{D \times M} \underbrace{\boldsymbol{\Sigma}_M}_{M \times M} \underbrace{\boldsymbol{V}_M^\top}_{M \times N} \in \mathbb{R}^{D \times N} \tag{10.51}$$

with orthogonal matrices $\boldsymbol{U}_M := [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_M] \in \mathbb{R}^{D \times M}$ and $\boldsymbol{V}_M := [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_M] \in \mathbb{R}^{N \times M}$ and a diagonal matrix $\boldsymbol{\Sigma}_M \in \mathbb{R}^{M \times M}$ whose diagonal entries are the M largest singular values of $\boldsymbol{X}$.

10.4.2 Practical Aspects

Finding eigenvalues and eigenvectors is also important in other fundamental machine learning methods that require matrix decompositions. In theory, as we discussed in Section 4.2, we can solve for the eigenvalues as roots of the characteristic polynomial. However, for matrices larger than 4×4 this is not possible because we would need to find the roots of a polynomial of degree 5 or higher. However, the *Abel-Ruffini theorem* (Ruffini, 1799; Abel, 1826) states that there exists no algebraic solution to this problem for polynomials of degree 5 or more. Therefore, in practice, we solve for eigenvalues or singular values using iterative methods, which are implemented in all modern packages for linear algebra.

Abel-Ruffini theorem

`np.linalg.eigh` or `np.linalg.svd`

In many applications (such as PCA presented in this chapter), we only require a few eigenvectors. It would be wasteful to compute the full decomposition, and then discard all eigenvectors with eigenvalues that are beyond the first few. It turns out that if we are interested in only the first few eigenvectors (with the largest eigenvalues), then iterative processes, which directly optimize these eigenvectors, are computationally more efficient than a full eigendecomposition (or SVD). In the extreme case of only needing the first eigenvector, a simple method called the *power iteration* is very efficient. Power iteration chooses a random vector $\boldsymbol{x}_0$ that is not in

power iteration

the null space of $\boldsymbol{S}$ and follows the iteration

$$\boldsymbol{x}_{k+1} = \frac{\boldsymbol{S}\boldsymbol{x}_k}{\|\boldsymbol{S}\boldsymbol{x}_k\|}, \quad k = 0, 1, \ldots . \tag{10.52}$$

This means the vector $\boldsymbol{x}_k$ is multiplied by $\boldsymbol{S}$ in every iteration and then normalized, i.e., we always have $\|\boldsymbol{x}_k\| = 1$. This sequence of vectors converges to the eigenvector associated with the largest eigenvalue of $\boldsymbol{S}$. The original Google PageRank algorithm (Page et al., 1999) uses such an algorithm for ranking web pages based on their hyperlinks.

If $\boldsymbol{S}$ is invertible, it is sufficient to ensure that $\boldsymbol{x}_0 \neq \boldsymbol{0}$.

10.5 PCA in High Dimensions

In order to do PCA, we need to compute the data covariance matrix. In D dimensions, the data covariance matrix is a $D \times D$ matrix. Computing the eigenvalues and eigenvectors of this matrix is computationally expensive as it scales cubically in D. Therefore, PCA, as we discussed earlier, will be infeasible in very high dimensions. For example, if our $\boldsymbol{x}_n$ are images with $10{,}000$ pixels (e.g., 100×100 pixel images), we would need to compute the eigendecomposition of a $10{,}000 \times 10{,}000$ covariance matrix. In the following, we provide a solution to this problem for the case that we have substantially fewer data points than dimensions, i.e., $N \ll D$.

Assume we have a centered dataset $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$, $\boldsymbol{x}_n \in \mathbb{R}^D$. Then the data covariance matrix is given as

$$\boldsymbol{S} = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top \in \mathbb{R}^{D \times D}, \tag{10.53}$$

where $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ is a $D \times N$ matrix whose columns are the data points.

We now assume that $N \ll D$, i.e., the number of data points is smaller than the dimensionality of the data. If there are no duplicate data points, the rank of the covariance matrix $\boldsymbol{S}$ is N, so it has $D - N + 1$ many eigenvalues that are 0. Intuitively, this means that there are some redundancies. In the following, we will exploit this and turn the $D \times D$ covariance matrix into an $N \times N$ covariance matrix whose eigenvalues are all positive.

In PCA, we ended up with the eigenvector equation

$$\boldsymbol{S}\boldsymbol{b}_m = \lambda_m \boldsymbol{b}_m, \quad m = 1, \ldots, M, \tag{10.54}$$

where $\boldsymbol{b}_m$ is a basis vector of the principal subspace. Let us rewrite this equation a bit: With $\boldsymbol{S}$ defined in (10.53), we obtain

$$\boldsymbol{S}\boldsymbol{b}_m = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{b}_m = \lambda_m \boldsymbol{b}_m . \tag{10.55}$$

We now multiply $\boldsymbol{X}^\top \in \mathbb{R}^{N \times D}$ from the left-hand side, which yields

$$\frac{1}{N}\underbrace{\boldsymbol{X}^\top \boldsymbol{X}}_{N \times N}\underbrace{\boldsymbol{X}^\top \boldsymbol{b}_m}_{=:\boldsymbol{c}_m} = \lambda_m \boldsymbol{X}^\top \boldsymbol{b}_m \iff \frac{1}{N}\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{c}_m = \lambda_m \boldsymbol{c}_m , \tag{10.56}$$

©2024 M. P. Deisenroth, A. A. Faisal, C. S. Ong. Published by Cambridge University Press (2020).

and we get a new eigenvector/eigenvalue equation: λ_m remains eigenvalue, which confirms our results from Section 4.5.3 that the nonzero eigenvalues of $\boldsymbol{X}\boldsymbol{X}^\top$ equal the nonzero eigenvalues of $\boldsymbol{X}^\top\boldsymbol{X}$. We obtain the eigenvector of the matrix $\frac{1}{N}\boldsymbol{X}^\top\boldsymbol{X} \in \mathbb{R}^{N\times N}$ associated with λ_m as $\boldsymbol{c}_m := \boldsymbol{X}^\top\boldsymbol{b}_m$. Assuming we have no duplicate data points, this matrix has rank N and is invertible. This also implies that $\frac{1}{N}\boldsymbol{X}^\top\boldsymbol{X}$ has the same (nonzero) eigenvalues as the data covariance matrix $\boldsymbol{S}$. But this is now an $N \times N$ matrix, so that we can compute the eigenvalues and eigenvectors much more efficiently than for the original $D \times D$ data covariance matrix.

Now that we have the eigenvectors of $\frac{1}{N}\boldsymbol{X}^\top\boldsymbol{X}$, we are going to recover the original eigenvectors, which we still need for PCA. Currently, we know the eigenvectors of $\frac{1}{N}\boldsymbol{X}^\top\boldsymbol{X}$. If we left-multiply our eigenvalue/eigenvector equation with $\boldsymbol{X}$, we get

$$\underbrace{\frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top}_{\boldsymbol{S}}\boldsymbol{X}\boldsymbol{c}_m = \lambda_m\boldsymbol{X}\boldsymbol{c}_m \tag{10.57}$$

and we recover the data covariance matrix again. This now also means that we recover $\boldsymbol{X}\boldsymbol{c}_m$ as an eigenvector of $\boldsymbol{S}$.

Remark. If we want to apply the PCA algorithm that we discussed in Section 10.6, we need to normalize the eigenvectors $\boldsymbol{X}\boldsymbol{c}_m$ of $\boldsymbol{S}$ so that they have norm 1. ♢

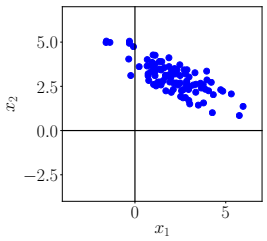
10.6 Key Steps of PCA in Practice

In the following, we will go through the individual steps of PCA using a running example, which is summarized in Figure 10.11. We are given a two-dimensional dataset (Figure 10.11(a)), and we want to use PCA to project it onto a one-dimensional subspace.

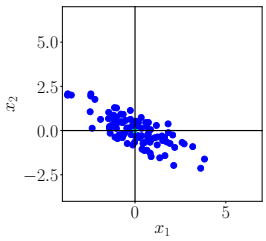
1. **Mean subtraction** We start by centering the data by computing the mean $\boldsymbol{\mu}$ of the dataset and subtracting it from every single data point. This ensures that the dataset has mean $\mathbf{0}$ (Figure 10.11(b)). Mean subtraction is not strictly necessary but reduces the risk of numerical problems.
2. **Standardization** Divide the data points by the standard deviation σ_d of the dataset for every dimension $d = 1, \dots, D$. Now the data is unit free, and it has variance 1 along each axis, which is indicated by the two arrows in Figure 10.11(c). This step completes the *standardization* of the data.

standardization

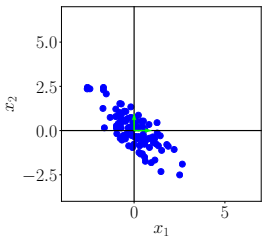
3. **Eigendecomposition of the covariance matrix** Compute the data covariance matrix and its eigenvalues and corresponding eigenvectors. Since the covariance matrix is symmetric, the spectral theorem (Theorem 4.15) states that we can find an ONB of eigenvectors. In Figure 10.11(d), the eigenvectors are scaled by the magnitude of the cor-



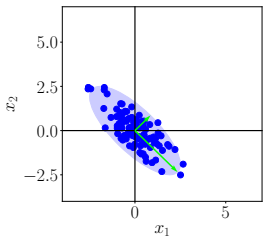
(a) Original dataset.



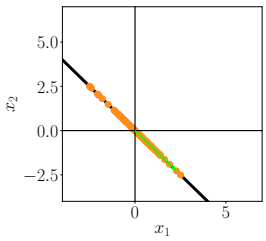
(b) Step 1: Centering by subtracting the mean from each data point.



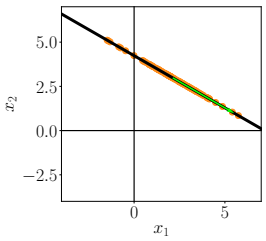
(c) Step 2: Dividing by the standard deviation to make the data unit free. Data has variance 1 along each axis.



(d) Step 3: Compute eigenvalues and eigenvectors (arrows) of the data covariance matrix (ellipse).



(e) Step 4: Project data onto the principal subspace.



(f) Undo the standardization and move projected data back into the original data space from (a).

Figure 10.11 Steps of PCA. (a) Original dataset; (b) centering; (c) divide by standard deviation; (d) eigendecomposition; (e) projection; (f) mapping back to original data space.

responding eigenvalue. The longer vector spans the principal subspace, which we denote by U. The data covariance matrix is represented by the ellipse.

4. **Projection** We can project any data point $\boldsymbol{x}_* \in \mathbb{R}^D$ onto the principal subspace: To get this right, we need to standardize $\boldsymbol{x}_*$ using the mean μ_d and standard deviation σ_d of the training data in the dth dimension, respectively, so that

$$x_*^{(d)} \leftarrow \frac{x_*^{(d)} - \mu_d}{\sigma_d}\,, \quad d = 1, \ldots, D\,, \tag{10.58}$$

where $x_*^{(d)}$ is the dth component of $\boldsymbol{x}_*$. We obtain the projection as

$$\tilde{\boldsymbol{x}}_* = \boldsymbol{B}\boldsymbol{B}^\top \boldsymbol{x}_* \tag{10.59}$$

with coordinates

$$\boldsymbol{z}_* = \boldsymbol{B}^\top \boldsymbol{x}_* \tag{10.60}$$

with respect to the basis of the principal subspace. Here, $\boldsymbol{B}$ is the matrix that contains the eigenvectors that are associated with the largest eigenvalues of the data covariance matrix as columns. PCA returns the coordinates (10.60), not the projections $\boldsymbol{x}_*$.

©2024 M. P. Deisenroth, A. A. Faisal, C. S. Ong. Published by Cambridge University Press (2020).